

Engineering Takeover Plan

An independent assessment of the delivered codebase, a reconciliation against the Phase 1–3 milestones, and a staged plan to reach production.

01 Executive summary

What I found when I took the codebase apart and ran it.

I have reviewed the full Photopol codebase, compiled the frontend, executed the backend test suite, and traced the critical paths by hand. My conclusion is that **the foundation is sound and worth keeping** — this is not a rebuild. Every feature ticked on the Phase 1–3 milestone list genuinely exists in code, and the underlying architecture is well-organised.

The gap is not in features. It is that **the milestone list measures code written, not a product proven to work**. Every item still outstanding is a live-service item — real Replicate, real Stripe, a real domain. To date nothing in Photopol has been verified against the paid services it depends on to function. Alongside that, my review surfaced six defects that the milestone list had no line item for, two of which have direct financial exposure.

BOTTOM LINE

We are closer to production than a rebuild, but further than the checklist implies. The remaining work is **verification and hardening, not feature development**. I estimate **one week** to a live, billable product.

02 Verified state of the build

Not taken on trust — each line below was executed or traced.

CHECK	RESULT	DETAIL
Frontend production build	PASS	30+ routes compile clean; TypeScript strict mode
Backend test suite	PASS	14 tests — but see coverage note below
Backend dependency install	FAIL	Conflict halts a clean checkout before it starts
Credit ledger correctness	SOUND	Reserve → settle, row locking, duplicate-charge protection
Payment webhook security	SOUND	Stripe and Paddle signatures both properly verified
Live AI (Replicate) end-to-end	NEVER RUN	Requires API token
Live payments end-to-end	NEVER RUN	Requires Stripe keys
Production deployment	NEVER RUN	No DNS, no TLS, no live host

Development history

The repository contains **two commits**. The first introduced 28,026 lines across 170 files in a single drop; the second was a follow-up refinement. The build specification was committed to the repository by mistake as `readme.txt`. There are no `TODO` or `FIXME` markers anywhere in the codebase, and the commit author is an unconfigured placeholder identity. This is consistent with a largely generated build, and it explains the shape of what I found: the architecture is orderly, while the operational layer — abuse control, deployment packaging, tests — is thin, because none of it was ever exercised under real conditions.

03 Defects found outside the milestone list

Ordered by business risk. All independently verified.

RISK	ISSUE	CONSEQUENCE
CRITICAL	Anonymous visitors can trigger unlimited paid AI jobs	Guest sessions are unauthenticated, unmetered and uncapped, and run against our paid Replicate account. An open-ended bill with no ceiling.
CRITICAL	Live user database committed to the repository	24 real accounts including administrator and demo password hashes are stored in version control, on a hosted Git remote. Those credentials must be rotated.
CRITICAL	Rate limiting protects the wrong endpoints	The limiter guards older routes while the main endpoint — the one that costs money on every request — is entirely unprotected. It is also per-process, so it stops working the moment we scale.
HIGH	Backend will not install from a clean checkout	An image-library version conflict blocks setup. Any new machine, CI runner or server hits this immediately.
HIGH	Deployment container runs the development server	The shipping container never performs a production build. Slower, heavier and unsuitable for live traffic.
HIGH	Background Removal silently uses the wrong engine	The manual tool bypasses the configured Replicate provider and falls back to a local library that no longer installs, so that path is dead. The guided outcome flow is unaffected.
MODERATE	Automated testing covers the oldest ~10% only	No coverage of the job engine, pipelines, billing or batch logic. A passing test run tells us very little.
MODERATE	Roughly 700 lines of orphaned code	Several unused interface files and two overlapping service layers doing the same work. Maintenance drag, not a live fault.

04 Milestone reconciliation

Measured against the agreed Phase 1–3 acceptance list.

PHASE	CODE DELIVERED	VERIFIED LIVE	OUTSTANDING
Phase 1 Core SaaS	COMPLETE	NONE	Live Replicate E2E · Live Stripe pack E2E · DNS & TLS deploy
Phase 2 Tools & queues	COMPLETE	NONE	Live subscription E2E · Production flags (queue, signed URLs, guest caps)
Phase 3 Product workspace	COMPLETE	NONE	Batch, export packs and product pipeline all confirmed as real implementations

I want to be fair to the work delivered: I checked the Phase 3 items specifically, and the provider failover, multi-format export, batch archive generation and marketplace size presets are genuine implementations rather than placeholders.

The build is honest about what was written.

The pattern that matters is in the third column. **Across all three phases, nothing has been validated against a live service.** The outstanding items are not minor leftovers — collectively they are the entire proof that the product functions commercially.

05 The plan from here

Four stages. Revenue-capable at the end of Stage 2.

STAGE 0 Takeover & control

1–2 days

- Transfer repository ownership, domain registrar, and all third-party accounts (Replicate, Stripe, storage) into accounts we control.
- Rotate the exposed administrator credentials and issue new API keys.
- Purge the committed database file and the build specification from version control history.
- Stand the project up locally end to end to establish a working baseline.

OUTCOME We own and can operate the asset.

STAGE 1 Stabilise & secure

2–3 days

- Fix the dependency conflict and remove the redundant machine-learning library — this alone materially reduces the deployment footprint.
- Correct the Background Removal engine routing so the tool uses the intended AI provider.
- Rebuild the deployment container to run a real production build.
- Move rate limiting onto the endpoints that actually cost money, backed by shared storage so it survives scaling.
- Introduce a hard ceiling on anonymous usage and enable the production safety flags.

OUTCOME The system installs cleanly and can no longer be used to run up an uncapped bill.

STAGE 2 Prove it live & launch

2–3 days

- Run the three outstanding end-to-end verifications: AI generation against live Replicate, a credit-pack purchase, and a subscription with the customer portal.
- Provision the production server, deploy the full stack with the background worker enabled, and configure private storage with signed URLs.
- Point photopol.us at the server, issue TLS certificates, and configure the payment webhooks against the live domain.
- Add automated coverage for the job engine and billing paths — the areas currently untested.
- Complete a live smoke test: sign up, edit an image, purchase credits, download the result.

OUTCOME Photopol is live, taking payments, and demonstrably works. **This is the point the project becomes commercially real.**

- Migrate the backend from Python to Node.js so the entire product runs on a single language and toolchain, matching the existing frontend.
- Port endpoint by endpoint, validating each against the live system as a working reference.
- Retire the duplicated service layer and remove the orphaned interface code.

OUTCOME One codebase, one language, materially cheaper to maintain and extend.

06 Stack and hosting decisions

Two recommendations, with the reasoning behind each.

Consolidating on Node.js — recommended, but scheduled after launch

I examined how tightly the backend is bound to Python. The answer is: barely at all. There is no scientific or machine-learning Python code in the project — the AI is entirely remote API calls. The backend is under 8,000 lines, and every component has a direct, mature Node equivalent, several of which are faster than what is in place today. **The migration is genuinely straightforward** and would leave us with one language across the whole product.

My recommendation is to do it in Stage 3 rather than now, for one reason: today we would be rewriting a system that has never been proven against live services, which makes it impossible to distinguish a migration bug from a pre-existing one. The highest-risk component to re-implement is the credit ledger, because it handles money — and it is currently correct. Migrating *after* launch means porting against a running reference we can test each endpoint against. Same total effort, considerably lower risk, and the product goes live three weeks sooner.

Hosting — a dedicated server, not serverless

Photopol runs image jobs that take minutes and needs a permanently running background worker alongside a database, a queue and file storage. That profile does not fit serverless hosting; the background worker in particular cannot run there at all, so a server would be required regardless. A modest VPS handles the entire stack for a predictable monthly cost, scales by adding worker capacity, and avoids per-request billing on long AI operations. I recommend provisioning it during Stage 2.

07 What I need to proceed

Stage 0 and Stage 1 can begin immediately. Stage 2 is blocked without these.

ITEM	BLOCKS	WHY
Replicate API token, funded	Stage 2	Required to verify AI generation works live
Stripe account & API keys	Stage 2	Required to verify payments and subscriptions
photopol.us registrar access	Stage 2	Required for DNS and TLS certificates
Approval for server provisioning	Stage 2	Small recurring hosting cost
Repository & account handover	Stage 0	Required to take ownership and rotate credentials

IMMEDIATE PRIORITY

The exposed administrator credentials and the uncapped anonymous AI usage are both live exposures today. I would like to address these first, ahead of any other work, regardless of how the wider plan is scheduled.

08 Timeline

Working days, assuming access is provided promptly.

STAGE	FOCUS	DURATION	MILESTONE
0	Takeover & control	1–2 days	Ownership secured, credentials rotated
1	Stabilise & secure	2–3 days	Clean install, cost exposure closed
2	Prove it live & launch	2–3 days	Live on photopol.us, taking payments
3	Consolidate the stack	2–3 weeks	Single-language codebase

Stages 0 to 2 together represent approximately **one working week** to a live, revenue-capable product. Stage 3 is a maintainability investment that can be scheduled to suit the business and does not gate launch.